

Criar conta

A tela aceita o que deveria recusar — e recusa pelo motivo errado

Hoje você não é o autor. Você é o testador.

terça-feira, 08/09/2026 · 18h às 22h · Laboratório de Informática

Unidade Curricular 11 · Executar teste e implantação de aplicativos computacionais

NOME: _____

SUMÁRIO

O que tem nesta apostila

As Partes 1 a 6 são a fundação: elas valem para a UC11 inteira, e não só para hoje. Guarde-a. E repare: TODO O CÓDIGO AQUI ESTÁ EM PORTUGOL — o C# vai para o quadro, e traduzir de um para o outro é metade do exercício.

Parte	O quê	Para quê serve
1	Por que testar	Erro, defeito e falha — e por que um programa que roda pode estar errado
2	O vocabulário	Caso de teste, critério de aceitação, teste estático e teste automatizado
3	Teste unitário por dentro	O que é, como funciona, para que serve, quando usar e quando não usar
4	Como se escreve um teste	Arrange-Act-Assert, o nome do teste, as cinco regras e o ciclo
5	A linha que confere	As três linhas da noite, o catálogo, e a tradução do Portugol para a ferramenta
6	A ferramenta	O projeto de teste no Visual Studio: o que já veio pronto e como se monta um
7	A regra do Cadastro	A especificação: o que cada método TEM de fazer
8	Mãos à obra	Os 18 casos do roteiro
9	Achou o vermelho, e agora	Descrever o achado, e comparar com o seu palpite
10	Socorro	Os três erros que aparecem em toda turma

PARTE

1

Por que testar

- Um programa que compila, roda — e mente
- Erro, defeito e falha: três palavras diferentes
- Quanto custa achar tarde
- Testar antes ou testar depois

O programa desta noite funciona

Dê dois cliques no Aula01Login.sln e aperte F5. A janela de criar conta abre, você preenche usuário e senha, clica no botão, e ela responde. Zero erro de compilação, zero aviso, nenhuma tela de exceção.

E ainda assim há coisa errada aí dentro. Não erro de digitação, não falta de ponto e vírgula: o compilador já teria pegado. O que está errado é mais difícil de ver, e é isto:

```
A REGRA diz: a garantia vale ate o 90o dia, inclusive
O CODIGO diz: if (dias < 90) return true;
A DIFERENCA: NaGarantia(90) devolve falso
```



Um caractere

É só o = que falta no <. Nenhuma ferramenta de compilação acusa isso, porque o código está sintaticamente perfeito. Só quem compara o código com a REGRA percebe — e é isso que se chama testar.

Este exemplo é de outro sistema, de propósito: no Cadastro desta noite, quem acha é você.

Erro, defeito e falha

No dia a dia a gente chama tudo de "bug". Em teste de software são três coisas diferentes, e elas acontecem em momentos diferentes.



1 • ERRO (ou engano)

Acontece na CABEÇA de quem escreve. A pessoa entendeu "a partir de 100" e digitou "maior que 100". É humano, e não dá para eliminar.



2 • DEFEITO

É o que o engano DEIXOU no código: a linha errada, parada no arquivo. Ela pode ficar ali meses sem que ninguém perceba.



3 • FALHA

É o que o USUÁRIO vê: o resultado errado na tela. Ela só acontece quando alguém passa exatamente pelo valor que acorda o defeito.



Defeito sem falha existe, e é o mais perigoso

O defeito da garantia está no código desde que ele foi escrito. Mas quem consulta no dia 30 ou no dia 45 nunca vê falha nenhuma. O defeito está lá o tempo todo; a falha esperava alguém consultar exatamente no dia 90.

A linha do tempo de um defeito

O defeito da garantia, visto de três lugares. Repare que o tempo entre a coluna 2 e a coluna 3 pode ser de meses — e que ninguém controla quem vai ser o usuário da coluna 3.

Momento	O que acontece	Quem percebe	Nome
Quinta, 14h	Quem escreve entende a regra errado	Ninguém	ERRO
Quinta, 14h02	A linha errada entra no arquivo	Ninguém	DEFEITO
Quinta, 14h05	O projeto compila com 0 erros	Ninguém	—
Sexta	A revisão do colega passa batido	Ninguém	—
Três meses depois	Um cliente consulta no dia 90	O cliente	FALHA
Três meses depois	O suporte abre um chamado	A empresa	—

O teste que você vai escrever hoje encurta essa linha do tempo para 14h06.

Quanto custa achar tarde

Achar um defeito custa dinheiro em qualquer fase. O que muda — e muda muito — é QUANTO. Quanto mais tarde, mais gente precisa se envolver para consertar a mesma linha de código.

Onde o defeito é achado	Quem precisa entrar	Custo relativo
Escrevendo o código	Só você	1
No teste, antes de entregar	Você e o testador	10
Na homologação com o cliente	+ gerente, + análise, + nova versão	100
Depois de o sistema estar no ar	+ suporte, + cliente parado, + reputação	1000



Esta tabela tem nome, e volta na Aula 3

Chama-se regra 10 de Myers, e volta na Aula 3: a cada fase que o defeito atravessa sem ser achado, o custo de consertar multiplica por dez. Não decore o número — entenda a direção da seta.

Testar antes ou testar depois

Existem duas maneiras de encaixar teste num projeto, e as duas têm nome no plano de curso. Nenhuma delas é "a certa".

Estratégia PREVENTIVA

Os testes são pensados JUNTO com os requisitos, antes de o código existir. Quem escreve o teste está perguntando "o que este método tem de fazer?" — e a resposta vira teste antes de virar código.

Custa mais no começo. Paga na coluna 1 da tabela anterior.

Estratégia REATIVA

Os testes são escritos DEPOIS que o código está pronto — normalmente porque alguém reclamou. É o que estamos fazendo hoje: a tela já existe, e nós chegamos depois.

É mais barata de começar. Paga na coluna 3 ou 4.

A nossa hoje é reativa por um motivo prático: para aprender a testar, é mais fácil começar por um programa que já existe e já está errado.

P A R T E

2

0 vocabulário

- Caso de teste, e por que ele é escrito antes
- Critério de aceitação
- Teste estático × teste dinâmico
- Teste manual × teste automatizado
- O que é TESTE UNITÁRIO

Caso de teste

Um caso de teste não é "vou lá e experimento". É um documento curto, escrito ANTES de rodar qualquer coisa, e ele tem sempre as mesmas partes:

Parte	O que é	Exemplo, para a garantia
Identificação	Um nome ou código que não se repete	CT-07
Pré-condição	O que precisa ser verdade antes	O projeto compila
Entrada	Os valores exatos que vão ser usados	dias = 90
Resultado esperado	O que TEM de acontecer, segundo a regra	verdadeiro
Resultado obtido	O que aconteceu de verdade	(preenche depois de rodar)
Situação	Passou / Não passou / Bloqueado	(idem)



O resultado esperado vem da REGRA, nunca do programa

Se você roda primeiro e escreve depois o que apareceu, o teste passa sempre — e não prova nada. O esperado sai da especificação, e é por isso que a Parte 7 desta apostila existe.

Critério de aceitação

Critério de aceitação é a resposta, escrita e combinada, para a pergunta: como sabemos que isto está pronto? Sem ele, "funcionando" é opinião.

Sem criterio de aceitacao, nao existe teste – existe palpite.

Sem critério

"A garantia tem que funcionar."

Funcionar como? No dia 90 ainda vale? E num número negativo? Duas pessoas leem isso e testam coisas diferentes.

Com critério

"NaGarantia(89) = verdadeiro. NaGarantia(90) = verdadeiro.
NaGarantia(91) = falso."

Agora dá para escrever quatro casos de teste, e duas pessoas escrevem os mesmos.

Teste estático × teste dinâmico

A primeira grande divisão do teste de software é esta, e ela é simples: o programa chegou a rodar?



ESTÁTICO — sem executar

Você lê. Confere o código contra a regra, olha o documento de requisitos, revisa com um colega.

Acha defeito ANTES de existir falha, e por isso é o mais barato de todos.

É o que você faz na primeira metade desta noite, no papel.



DINÂMICO — executando

Você roda. Digita valores, chama o método, e compara o que saiu com o que devia sair.

Acha o que passa despercebido na leitura, e prova o achado.

É o que você faz na segunda metade, com o Test Explorer.



Os dois, e nessa ordem

Ler primeiro e rodar depois é o que torna a comparação do fim da noite possível: você anota o que ACHOU lendo, e depois vê o que a barra PROVOU.

Teste manual × teste automatizado

Um teste manual é uma pessoa seguindo um roteiro. Um teste automatizado é um programa que testa outro programa. Os dois são teste de verdade; o que muda é o que acontece na SEGUNDA vez.

	Manual	Automatizado
Primeira execução	Rápido de começar	Custa escrever o código
Décima execução	Custa o mesmo de novo	Custa um clique
Depois de uma correção	Alguém precisa lembrar de refazer	Roda tudo de novo sozinho
Erra?	Cansa, pula passo, desatenção	Faz exatamente igual sempre
Acha coisa nova?	Sim — a pessoa desconfia	Não. Só confere o que foi escrito



Automatizado não substitui manual

O teste automatizado nunca vai desconfiar de nada — ele só confere o que alguém escreveu nele. Quem desconfia é você. A máquina só garante que a desconfiança de hoje continue sendo conferida daqui a seis meses.

P A R T E

3

Teste unitário por dentro

- O que é uma unidade, e o que é um caso de teste
- Como a ferramenta descobre e roda os seus testes
- Para que serve, quando usar — e quando não usar
- O que a barra verde NÃO prova

O que é teste unitário

Teste unitário é o teste da MENOR peça do programa que dá para conferir sozinha. Em C#, essa peça é o método.

Uma unidade = um metodo. Um caso de teste = um comportamento dele.

"Unitário" não quer dizer pequeno nem fácil. Quer dizer ISOLADO: quando o teste do SenhaPreenchida fica vermelho, o problema está no SenhaPreenchida — não na tela, não no banco, não no teclado. Você já sabe onde procurar antes de começar a procurar.

Um método

Cadastro.SenhaPreenchida("abc")

Um comportamento

"senha com texto devolve verdadeiro"

Um teste

Um teste, uma linha de conferência

Os níveis de teste, e onde o unitário fica

O unitário é o primeiro de quatro níveis. Hoje só o primeiro; os outros três aparecem na Aula 4, quando houver o que integrar.

Nível	O que se testa	Exemplo no Cadastro	Hoje?
Unitário	Um método, isolado	SenhaPreenchida("abc") devolve verdadeiro	SIM
Integração	Duas peças conversando	O PodeCriarConta chamando os outros seis	em parte
Sistema	O programa inteiro, como o usuário usa	Abrir a janela, preencher os três campos, clicar	não
Aceitação	O cliente decide se aceita	O professor aceita a entrega	não

Repare no "em parte" da integração: o PodeCriarConta não decide nada sozinho, ele só pergunta para outros seis métodos. Testar o PodeCriarConta é testar sete peças juntas — e é por isso que ele é o mais interessante dos dez.

Um programa que roda o outro programa

Vale dizer com todas as letras: o projeto de teste é UM PROGRAMA. Tem .csproj, compila, vira executável. O que ele faz quando roda é chamar os métodos do OUTRO programa e comparar o que voltou.

O teste nao le a sua mente e nao le o seu codigo. Ele CHAMA e COMPARA.

#	O que acontece quando você manda executar	Quem faz
1	Os dois projetos são compilados. Se um dos dois não compilar, nada roda	O compilador do C#
2	O adaptador varre o programa de teste atrás das classes de teste, e dentro delas dos métodos marcados como teste	MSTest.TestAdapter
3	Cada método encontrado roda sozinho, do começo ao fim, numa instância nova da classe	MSTest.TestFramework
4	Cada um recebe um resultado: passou, falhou ou nem rodou. O placar é a soma disso	O Test Explorer

O passo 2 explica um susto clássico: método sem a marca de teste em cima não existe para o adaptador. Não dá erro, não aparece na lista, não conta no placar.

Cinco motivos, e nenhum deles é "porque o professor pediu"

Escrever teste dá trabalho — são linhas que o cliente nunca vê. O que se compra com esse trabalho é isto:

O que ele dá	Na prática
Acha cedo	Um defeito achado no seu computador custa uma fração do mesmo defeito achado pelo cliente. É a conta da Parte 1.4
Documenta a regra	O teste NaGarantia_NoUltimoDia_DevolveVerdadeiro diz o que o sistema promete — e, ao contrário do comentário, não fica desatualizado sem ficar vermelho
Protege de regressão	Você mexe numa regra, roda tudo, e em segundos sabe se quebrou outra. Sem isso, mexer em código que funciona é aposta
Repete de graça	Dezoito casos rodam em dois segundos. Os mesmos dezoito conferidos à mão, pelo menu, levam vinte minutos — e ninguém faz isso duas vezes por dia
Aponta o lugar	O teste do SenhasConferem ficou vermelho: o problema está no SenhasConferem. Você começa a procurar já sabendo onde

Três dos cinco só aparecem DEPOIS, quando alguém mexer no código. É por isso que teste parece desperdício na primeira semana e vira indispensável no segundo mês.

O que merece um teste unitário

Nem toda linha precisa de teste. O critério é simples: teste unitário serve onde existe uma REGRA que pode ser violada em silêncio.

Situação	Vale?	Por quê
O método faz uma conta, uma validação ou uma decisão	SIM	É onde mora a regra, e é onde um caractere errado passa despercebido
Você vai corrigir um defeito	SIM	Escreva primeiro o teste que reproduz o defeito: ele fica vermelho, a correção o deixa verde, e ele fica de guarda para o defeito não voltar
Você vai mexer em código que já funciona	SIM	Escreva os testes ANTES de mexer. Eles viram a rede: se ficarem vermelhos depois, você quebrou algo
O método só guarda ou devolve um valor, sem decidir nada	não compensa	Testar get e set é testar o compilador. Gasta linha e não acha defeito
O código ainda vai mudar de forma nos próximos dias	espere	Você reescreveria o teste junto. Teste se paga quando o formato do método estabiliza

Os dez métodos do Cadastro.cs caem todos na primeira linha da tabela.

O que não dá para testar assim — e o que se usa no lugar

Teste unitário precisa de duas coisas: o método tem que DEVOLVER algo, e tem que devolver SEMPRE o mesmo diante da mesma entrada. Faltando uma das duas, a ferramenta certa é outra.

O que o método faz	Por que o teste unitário não alcança	O que se usa
Escreve na tela ou lê do teclado	Não devolve nada. Não há o que comparar	Teste de sistema
Grava ou lê de um banco de dados	O resultado depende do que já estava gravado	Teste de integração
Chama um serviço pela internet	Depende de um computador que não é seu, e que pode estar fora do ar	Teste de integração
Usa DateTime.Now	O mesmo teste passa hoje e falha amanhã. Não é repetível	Receber a data por parâmetro
Usa Random	Cada execução dá um número. Não dá para prever o esperado	Receber o sorteio por parâmetro



O truque é sempre o mesmo

Nas duas últimas linhas, repare na solução: o que vem de fora ENTRA POR PARÂMETRO. Um método que recebe a data em vez de perguntar as horas ao sistema volta a ser testável, e continua funcionando igual no programa de verdade — que passa DateTime.Now na hora de chamar.

Por que a classe de regra não pode ter Console

Um mês atrás, na UC12, este comentário foi escrito no topo do Calculo.cs. O Cadastro.cs de hoje repete a mesma frase. Parecia uma regra de arrumação. Hoje ele cobra o preço:

```
1 /// Procure por C-o-n-s-o-l-e aqui dentro. Voce nao vai achar,  
2 /// e e de proposito. Nenhum metodo le do teclado e nenhum  
3 /// metodo escreve na tela.  
4 ///  
5 /// E por isso que da para conferir cada um deles sem abrir  
6 /// o programa.
```

Um método que ESCRIBE na tela

Só pode ser conferido por alguém olhando a tela. Nenhum programa consegue dizer se o texto que apareceu estava certo.

Um método que DEVOLVE valor

Recebe por parâmetro, devolve com return. Outro programa chama, guarda o que voltou, e compara. É testável.

É por isso que hoje só o Cadastro.cs está sob teste.

Três coisas que o verde não prova

Esta página existe para você não sair daqui com fé demais na barra verde. Teste é evidência, não certificado.

Verde não é "o programa está certo"

É "o que eu conferi está certo". Um método sem nenhum teste tem exatamente zero vermelhos — e pode ser o pior do sistema.

Cobertura não é qualidade

Dá para passar por 100% das linhas sem conferir nada, se os Assert forem frouxos. Cobertura mede o que foi EXECUTADO, não o que foi VERIFICADO.

O teste também pode estar errado

Se o esperado que você digitou não é o que a regra manda, o vermelho acusa o programa por um crime do teste. Confira o esperado contra a especificação.

Testar mostra a PRESENÇA de defeitos. Nunca mostra a AUSENCIA deles.

A frase é de Edsger Dijkstra, de 1970, e continua valendo. É o motivo de o trabalho do testador ser escolher BEM o que procurar — assunto de amanhã à noite.

As palavras que aparecem daqui para a frente

Nenhuma é decoreba: todas voltam nas próximas noites. Esta página é para consultar.

Palavra	O que quer dizer
Caso de teste	Uma entrada, uma ação e um resultado esperado. É a unidade de trabalho do testador
Asserção (Assert)	A linha que compara o esperado com o obtido. É ela que decide verde ou vermelho
Suíte de testes	O conjunto de testes que rodam juntos. A sua suíte de hoje terá 18 casos
Fixture	O cenário montado antes de o teste rodar — os objetos e valores de que ele precisa
Massa de teste	Os dados escolhidos para alimentar os casos. Escolher bem é o assunto da Aula 2
Cobertura	A porcentagem do código executada pelos testes. Mede alcance, não qualidade
Regressão	Quando algo que funcionava para de funcionar depois de uma mudança
Teste de confirmação	Rodar de novo o teste que estava vermelho, para confirmar que a correção resolveu
Dublê (mock)	Um objeto de mentirinha que finge ser o banco ou a internet para o teste rodar sozinho

PARTE

4

Como se escreve um teste

- Arrange, Act, Assert — e a forma curta de uma linha só
- A anatomia de um teste, parte por parte
- O nome do teste é a mensagem de erro
- As cinco regras, e o ciclo: vermelho, conserta, verde, roda tudo

Arrange • Act • Assert

Todo teste do mundo — em C#, em Java, em Python — tem a mesma estrutura de três partes. Ela tem nome em inglês, e você vai ver essa sigla a vida inteira: AAA.

```
1 TESTE  naGarantia_NoUltimoDia_DevolveVerdadeiro
2 inicio
3     // ARRANGE - preparo o cenario
4     dias <- 90
5     // ACT      - executo o metodo que quero conferir
6     resultado <- naGarantia(dias)
7     // ASSERT   - comparo o obtido com o esperado
8     conferir_verdadeiro(resultado)
9 fimteste
```

ARRANGE • preparar

Os valores de entrada e o cenário. Aqui não se confere nada — só se monta.

ACT • agir

Uma única chamada: o método sob teste. Guarde o que ele devolveu.

ASSERT • conferir

A comparação. É a única linha que pode reprovar o teste.

Quando as três partes viram uma linha só

Quando o método recebe valores simples e devolve na hora, dá para escrever as três partes numa linha. É o que faremos a noite inteira, e é legítimo:

```
1 // as tres partes, uma embaixo da outra
2 dias      <- 90
3 resultado <- naGarantia(dias)
4 conferir_verdadeiro(resultado)
5
6 // as tres partes, na mesma linha
7 conferir_verdadeiro( naGarantia(90) )
```

Na linha de baixo, o arranjo é o 90 (entra direto na chamada), a ação é o `naGarantia(90)`, e a conferência é o `conferir_verdadeiro` que recebe o que voltou. As três partes continuam lá — só não têm mais uma linha cada.



Quando voltar para a forma longa

Assim que o cenário precisar de duas ou três linhas de preparo, volte ao formato da página anterior. Uma linha só é atalho, não regra.

A anatomia de um teste

```
1 TESTE  naGarantia_NoUltimoDia_DevolveVerdadeiro
2 inicio
3     conferir_verdadeiro( naGarantia(90) )
4 fimteste
```

Parte	Por que ela existe
A marca de TESTE	É ela que faz o Test Explorer ENXERGAR o método. Sem ela o teste não aparece na lista e nunca roda — e não dá erro nenhum. No quadro você vai ver como ela se escreve em C#
inicio / fimteste	Um teste não devolve nada e não recebe nada. Quem responde é a linha de conferência lá dentro
O nome	Vira a linha da lista quando o teste fica vermelho. Por isso ele conta a REGRA, e não "teste1"
conferir_...	A única linha que decide. Sem ela o teste passa sempre — e não prova nada

O padrão do nome: O_QUE_Testa _ ComQualEntrada _ OQueDeveDar.

O nome é a mensagem de erro

O nome do método de teste não é enfeite: é o texto que aparece na lista do Test Explorer quando ele fica vermelho. Use sempre três pedaços:

Metodo _ Cenario _ ResultadoEsperado

Assim não	Assim sim	O que o nome bom conta
teste1	NaGarantia_NoUltimoDia_DevolveVerdadeiro	qual método, qual caso, o que a regra promete
TesteGarantia	NaGarantia_UmDiaDepois_DevolveFalso	que o caso é o dia seguinte ao limite
TestaValor	ValorDoReparo_DentroDaGarantia_NaoCobra	que a expectativa é não cobrar nada
Metodo2Ok	ValorDoReparo_ForaDaGarantia_CobraIntegral	que fora da garantia o valor é cheio

Nome longo em teste não é defeito: é documentação. Se o nome for teste1, a lista vermelha não informa nada e você abre o código para descobrir o que quebrou.

As cinco regras de um teste que se paga

Um teste mal escrito é pior do que nenhum: dá confiança falsa, ou fica vermelho sem motivo até alguém desligar a suíte inteira. Cinco regras evitam os dois destinos.

Regra	O que significa	Se você quebrar
Rápido	A suíte inteira roda em segundos	Ninguém roda antes de entregar, e a rede de proteção deixa de existir
Independente	Cada teste monta o próprio cenário. O MStest ajuda: cria uma instância nova da classe a cada método	A ordem passa a importar; um vermelho derruba dez, e você não sabe qual é o de verdade
Repetível	Mesma entrada, mesmo resultado — hoje, amanhã e na máquina do colega	O teste vira loteria, e o time aprende a ignorar vermelho
Autoverificável	Ele mesmo diz passou ou falhou, sem alguém conferir número na tela	Voltou o trabalho manual que o teste existia para eliminar
Um conceito por teste	Um comportamento por método de teste	O primeiro Assert que falha impede os de baixo de rodar, e você fica sem saber o resto

Vermelho, conserta, verde, roda tudo de novo

Teste não é uma atividade que termina: é um ciclo. Estes quatro passos são o trabalho do testador, e as quatro noites desta semana são quatro voltas nele.

#	Passo	Nome técnico	Onde na semana
1	Escrever o teste a partir da REGRA e vê-lo ficar vermelho	teste dinâmico	hoje
2	Ler a mensagem e descrever o achado: regra, código, diferença	relatório de defeito	hoje e quinta
3	Corrigir o código e rodar de novo AQUELE teste até ficar verde	teste de confirmação	quinta
4	Rodar a suíte INTEIRA, para ver se a correção quebrou outra coisa	teste de regressão	quinta e sexta



O passo 4 é o que quase todo mundo pula

É tentador parar no 3: o teste que estava vermelho ficou verde, missão cumprida. Na quinta você vai ver ao vivo o que isso custa. Hoje a noite para no passo 2, de propósito: ninguém corrige nada. Achar e descrever já é trabalho suficiente para a primeira noite.

O preparo que roda antes de cada teste

Quando vários testes precisam do MESMO preparo, o MSTest tem dois atributos para isso:

```
1 CLASSE DE TESTE  GarantiaTests
2 inicio
3     produto: Produto
4
5     ANTES DE CADA TESTE
6         produto <- novo Produto("Furadeira", 250.00)
7     fim
8
9     DEPOIS DE CADA TESTE
10        produto <- nulo
11    fim
12 fimclasse
```



Rodam a cada teste — e hoje você não usa nenhum dos dois

Eles rodam a cada teste, não uma vez só: é assim que o MSTest garante a independência. Os métodos do Cadastro.cs são static e não guardam estado, então não há o que preparar nem limpar. Estão aqui porque você vai encontrá-los em qualquer projeto de verdade.

PARTE

5

A linha que confere

- Conferir verdadeiro e conferir falso — as duas da noite de hoje
- Conferir igual: o esperado vem primeiro
- A armadilha do double, para quinta e sexta
- O catálogo completo, e a tradução para a ferramenta

Conferir igual

```
1 //          esperado          obtido
2 //          |                  |
3 conferir_igual( 3 , diasDeGarantiaQueFaltam(87) )
```

A ordem importa para a MENSAGEM, não para o resultado. Se você inverter, o teste continua passando ou falhando igual — mas a mensagem vai dizer "Expected 2, Actual 1" quando era o contrário, e você vai procurar defeito no lugar errado.

**Esperado primeiro. Sempre.**

É a convenção do MSTest, do JUnit e de praticamente toda ferramenta de teste que existe. Vale a pena decorar de uma vez.

A armadilha do double

Esta página NÃO é do Cadastro desta noite — aqui não há um único número com casa decimal. Ela é para quinta e sexta, quando aparecerem dinheiro e média. Leia agora e volte quando precisar: escreva este teste e rode, e ele fica VERMELHO sem que haja nada de errado no programa.

```
1 soma <- 0.1 + 0.2
2 conferir_igual(0.3, soma)
```

```
1 Expected:<0.3>.   Actual:<0.300000000000000004>.
```

Não é bug do C#, e não é defeito de programa nenhum. O tipo double guarda o número em base 2, e 0.1 em base 2 é uma dízima — igual a 1/3 em base 10, que não cabe em nenhuma quantidade de casas. Isso acontece em toda linguagem de programação que usa ponto flutuante.



Anotar isso como defeito é o erro mais caro da semana

Você gasta a noite procurando o que consertar num método que está certo. A próxima página resolve — e na quinta-feira ela vale ouro.

A FOLGA — o terceiro parâmetro

A conferência de igual entre double aceita um terceiro valor: a folga (delta). Ela diz "aceito como igual se a diferença for menor que isto".

```
1 FOLGA <- 0.0001
2
3 conferir_igual(0.3, soma, FOLGA) // verde
```

Comparou double? Poe a FOLGA. Toda vez.

Leva folga

double, decimal, float — tudo que tem casa decimal. No Cadastro desta noite: nenhum método.

NÃO leva folga

SIM/NÃO, número inteiro, texto. Aqui a comparação é exata, e a folga nem existe como parâmetro. São estes os três tipos da noite de hoje.

As três linhas da noite

Oito dos dez métodos do Cadastro respondem SIM ou NÃO, um devolve um número e um devolve um texto. Não há double nenhum, então a noite inteira cabe em três linhas. A escrita exata em C# vai para o quadro:

Quando o método devolve...	Em Portugal, escreva	Na ferramenta
SIM ou NÃO, e você espera SIM	conferir_verdadeiro(obtido)	IsTrue
SIM ou NÃO, e você espera NÃO	conferir_falso(obtido)	IsFalse
um número	conferir_igual(esperado , obtido)	AreEqual
um texto	conferir_igual(esperado , obtido)	AreEqual



Um Assert por teste — e a frase de texto vem da constante

Dá para pôr várias conferências num mesmo teste, mas não faça isso hoje: se a primeira falhar, as de baixo nem rodam.

E as três frases do MensagemDoErro já vêm declaradas como constante no topo do seu arquivo. Use elas, nunca o texto digitado à mão: um ponto final esquecido produz um vermelho que não é do programa.

O catálogo completo do MSTest

Guarde esta página: ela vale para as quatro noites. A escrita completa de cada um sai no quadro.

O nome	Fica verde quando	Hoje?
AreEqual	os dois valores são iguais — número, texto, SIM/NÃO, caractere	sim, muito
AreEqual com FOLGA	os dois double estão a menos de FOLGA um do outro	quinta e sexta
AreNotEqual	o obtido é qualquer coisa MENOS aquele valor	raro
IsTrue	a condição é verdadeira	sim
IsFalse	a condição é falsa	sim
IsNull	a referência está vazia: nenhum objeto foi criado	não hoje
IsNotNull	existe um objeto ali — o primeiro teste de quem devolve objeto	não hoje
AreSame	as duas variáveis apontam para o MESMO objeto na memória	não hoje
IsInstanceOfType	o objeto é do tipo que você disse	não hoje
ThrowsException	o trecho indicado LANÇOU a exceção esperada	não hoje
Fail	nunca: reprova na hora. Marca um caminho proibido	não hoje
StringAssert • CollectionAssert	para texto (Contains, StartsWith) e listas (AreEqual, Contains)	na Aula 4

Do Portugol para a ferramenta

O C# de verdade vai para o QUADRO, e traduzir é o exercício. Aqui está o mapa dos nomes; a escrita exata você copia de lá.

No Portugol desta apostila	Na ferramenta, é o	O que ele faz
TESTE nome	atributo de método de teste	faz o Test Explorer enxergar o método
CLASSE DE TESTE nome	atributo de classe de teste	diz que a classe guarda testes
conferir_verdadeiro(x)	IsTrue	verde quando x é verdadeiro
conferir_falso(x)	IsFalse	verde quando x é falso
conferir_igual(esp , obt)	AreEqual	verde quando os dois são iguais
ANTES / DEPOIS DE CADA TESTE	atributos de preparo e limpeza	rodam em volta de cada teste
x <- 90	atribuição	guarda o valor na variável



Por que não já vem escrito

Ler código pronto e escrever código são coisas diferentes — só a segunda fica.

PARTE

6

A ferramenta: MSTest

- Onde o MSTest se encaixa entre as ferramentas do mercado
- O seu projeto de teste já vem montado — e como ele foi montado
- A referência de projeto — a linha que faz tudo funcionar
- A anatomia de um teste
- O Test Explorer: a barra verde e a barra vermelha

As ferramentas de teste, e o que cada uma faz

O plano de curso cita Mantis, TestLink, JUnit e Selenium. Elas não competem entre si — fazem coisas diferentes. Vale saber quem é quem antes de usar a nossa.

Ferramenta	Categoria	O que ela faz	Serve aqui?
JUnit	Execução de teste unitário	Roda testes de código Java e diz o placar	É a de Java
MSTest	Execução de teste unitário	O mesmo, para .NET. Já vem no Visual Studio	É a NOSSA
NUnit / xUnit	Execução de teste unitário	Alternativas ao MSTest, em .NET	Equivalentes
Selenium	Automação de navegador	Clica e digita num site sozinho	Só para web
TestLink	Gestão de casos de teste	Guarda o roteiro e o histórico de execução	Não executa
Mantis	Gestão de defeitos	Guarda o chamado do defeito e o ciclo de vida dele	Não executa



Por que MSTest e não os outros

Zero instalação: ele já vem no Visual Studio 2022, e o projeto de teste é um template da própria IDE. Não precisa de servidor, não precisa de conta, e depois da primeira compilação roda sem internet.

1

6.2 • O RITUAL

Como o projeto de teste foi montado

HOJE VOCÊ NÃO PRECISA FAZER ISTO: o Aula01Login.sln já vem com o projeto de teste montado e ligado. Leia agora, execute na Aula 2.

#	No Visual Studio	O que escolher
1	Com a solução aberta: clique com o botão direito na SOLUÇÃO (a linha de cima do Gerenciador de Soluções)	Adicionar → Novo Projeto
2	Na caixa de busca do assistente	digite: mstest
3	Na lista	Projeto de Teste do MSTest · C#
4	Nome do projeto	LoginTestes
5	Local	a MESMA pasta onde está a login-defeituoso
6	Framework	net8.0 — o mesmo do projeto testado



A primeira compilação precisa de internet — uma vez só

Vale para o seu também: o MSTest traz três pacotes do NuGet na PRIMEIRA compilação. Depois disso ficam no cache e rodam sem rede. Se aparecer "pacote não encontrado", chame o professor: não é você.

2

6.3 • A LINHA QUE FAZ TUDO FUNCIONAR

A referência de projeto

O projeto de teste nasce sem enxergar a tela. Para chamar `Cadastro.UsuarioPreenchido` ele precisa de uma REFERÊNCIA. No seu `.sln` ela já está feita — abaixo, como se faz.

#	No Visual Studio	O que escolher
1	Botão direito no projeto <code>LoginTestes</code> → Adicionar	Referência de Projeto
2	Na lista de projetos da solução	marque <code>Login</code>
3	Confirme	OK

Isso escreve UMA LINHA dentro do `LoginTestes.csproj` — a linha que diz qual projeto este aqui usa. Abra o arquivo e procure por ela: está lá embaixo, dentro do último bloco, e é a única do arquivo que cita uma pasta.



Guarde esta linha

Trocar a pasta que ela cita por outra que também tenha um `Cadastro.cs` faz a MESMA suíte rodar contra outro programa, sem mudar um teste. Chama-se parametrização, e volta na Aula 2.

3

6.4 • ONDE OS TESTES MORAM

As duas ligações, e o primeiro arquivo

No seu projeto o CadastroTests.cs já está lá, com o primeiro teste pronto e dezessete para você escrever. Ele começa assim:

```
1 usar    a biblioteca de testes
2 usar    o projeto que vai ser testado
3
4 CLASSE DE TESTE CadastroTests
5 inicio
6     // os dezoito testes moram aqui dentro
7 fimclasse
```

a biblioteca de testes

Traz a marca de teste e a linha que confere. Sem ela, nada roda.

o projeto que vai ser testado

Traz a classe Cadastro. Sem ela, erro CS0246.

O Test Explorer

Menu Testar → Gerenciador de Testes (em inglês, Test → Test Explorer). Deixe essa janela aberta a noite inteira — é ela que responde.

O que fazer	Como
Rodar todos os testes	O botão  no alto da janela, ou Ctrl+R depois Ctrl+A
Rodar um só	Botão direito no nome dele → Executar
Ver por que falhou	Clique no teste vermelho: a mensagem aparece no painel de baixo
Ir para o código do teste	Duplo clique no nome
A lista não aparece?	Compile a solução primeiro (Ctrl+Shift+B). O Test Explorer só lista o que já compilou



Compilar não é rodar

Compilar (Ctrl+Shift+B) faz os testes APARECEREM na lista. Rodar é outro botão. Escreveu um teste novo e ele não apareceu? Compile.

A barra verde e a barra vermelha

✓ VERDE — passou

O valor que o método devolveu é o valor que o Assert esperava.

ATENÇÃO: verde não quer dizer "o programa está certo". Quer dizer "o que EU conferi está certo". O que você não escreveu, ninguém conferiu.

✗ VERMELHO — não passou

O valor devolvido é diferente do esperado.

Um vermelho não acusa ninguém: ele diz que a REGRA e o CÓDIGO discordam. Pode ser o código que está errado — ou a sua conferência.

O terceiro estado é o CINZA — teste que nem chegou a rodar, normalmente porque a solução não compila. Cinza não é passou nem falhou: é "não sei".

Verde e a ausencia de prova, nao a prova de ausencia.

Ler a mensagem de falha

Quando um teste fica vermelho, o painel de baixo do Test Explorer mostra sempre a mesma estrutura. Aprenda a ler estas duas palavras e você economiza a noite:

```
1 A conferencia de VERDADEIRO falhou.  
2 Expected:<True>. Actual:<False>.  
3  
4 em naGarantia_NoDia90_AindaVale  
5 linha 118
```

Expected = o que a REGRA manda

É o primeiro valor da sua conferência. Se ele estiver errado, o problema é do teste, não do programa.

Actual = o que o CÓDIGO devolveu

É o que o método realmente calculou. A diferença entre os dois é o tamanho exato do defeito.

Anote os dois números. Eles vão para a folha de achados, na Parte 9.

PARTE

7

A regra do Cadastro

- A especificação: o que cada método TEM de fazer
- Como ler o código contra a regra
- A folha de palpites

A especificação, método a método

Método	A regra	Um caso que vale a pena
UsuarioPreenchido(u)	o usuário é obrigatório. Texto vazio não serve	"joao" serve, "" não
SenhaPreenchida(s)	a senha é obrigatória. Texto vazio não serve	mesma ideia da linha de cima
UsuarioDisponivel(u)	já existem TRÊS contas: ana, bruno e carla. Disponível é quem NÃO é nenhuma das três	"joao" está livre
SenhasConferem(s, r)	a senha e a repetição têm de ser EXATAMENTE iguais — mesmo texto, mesma ordem	não basta terem o mesmo tamanho
SenhaProibida(s)	QUATRO senhas são proibidas: senha, admin, qwerty e senac. Devolve verdadeiro quando é uma delas	"girassol42" não é proibida
CaracteresIguaisSeguidos(s)	o tamanho da MAIOR sequência de iguais seguidos	"abc"=1 • "aab"=2 • "ab111"=3
SemRepeticaoDemais(s)	três iguais seguidos já é demais. Com 2 passa; com 3 não	pergunta para o método de cima
SenhaDiferenteDoUsuario(u, s)	a senha não pode ser igual ao nome de usuário	usuário joao com senha joao: não
PodeCriarConta(u, s, r)	TODAS as sete regras ao mesmo tempo. Não decide sozinho: pergunta para os outros	basta uma falhar para recusar
MensagemDoErro(u, s, r)	diz POR QUE recusou. O mais forte manda, nesta ordem: campo em branco • usuário em uso • senhas diferentes • senha proibida • repetição • senha igual ao usuário	dando tudo certo, texto vazio

Como ler o código contra a regra

Você lê o Cadastro.cs impresso, um método por vez, e faz sempre as mesmas cinco perguntas. Elas não dizem ONDE está o defeito — dizem o que procurar:

#	A pergunta	Por que ela pega defeito
1	A regra cita uma LISTA e diz quantos itens ela tem? Conte os itens no código. Estão todos lá?	Falta um, e só quem testar aquele valor descobre
2	A regra manda comparar o CONTEÚDO de dois textos? O código compara o conteúdo, ou só o tamanho?	Dois textos do mesmo tamanho podem ser completamente diferentes
3	A regra tem VÁRIAS condições que valem AO MESMO TEMPO? O código usa && ou usa ?	Trocar E por OU não quebra nada — só faz o método parar de recusar
4	Tem laço percorrendo o texto? Ele vai até o ÚLTIMO caractere, ou para um antes?	Uma volta a menos só aparece quando a resposta está no fim
5	A regra tem uma ORDEM de prioridade? O código confere na mesma ordem?	A resposta pode estar certa e a explicação, errada



Anote o palpite mesmo sem certeza

Você vai comparar a sua lista com a barra no fim da noite. Palpite errado não vale menos que palpite certo — os dois ensinam, e é exatamente por isso que a folha existe.

Folha de palpites

Uma linha por suspeita. Só as duas primeiras colunas agora; a terceira você preenche depois de rodar os testes.

#	Método suspeito	Por quê (regra × código)	A barra confirmou?
1			☐ sim ☐ não
2			☐ sim ☐ não
3			☐ sim ☐ não
4			☐ sim ☐ não
5			☐ sim ☐ não
6			☐ sim ☐ não
7			☐ sim ☐ não
8			☐ sim ☐ não

Oito linhas não quer dizer oito defeitos. É só o tamanho da folha.

PARTE

8

Mãos à obra — os 18 casos

- O primeiro sai pronto: é o molde
- Dezesete TODO, um Assert em cada
- Não suponha quantos vão ficar vermelhos

0 roteiro de testes

No Gerenciador de Soluções, abra LoginTestes e dê dois cliques no CadastroTests.cs. Os 18 casos já estão lá, cada um com NOME e com a REGRA escrita no comentário de cima. O corpo é que está vazio — e é ele que você vem escrever.

Um Assert em cada. Não suponha quantos vão ficar vermelhos.

Esse roteiro pronto não é preguiça nossa: no trabalho de verdade, quem executa o teste quase nunca é quem escreveu o roteiro. Executar o roteiro de outra pessoa, exatamente como está escrito, é metade do trabalho de um testador.



A tentação de contar

Alguém vai perguntar quantos defeitos existem. Não vamos dizer — e não adianta contar os TODO, porque a maioria deles passa. Escreva os 18, rode, e deixe a barra responder.

1

0 molde — este já vem escrito

```
1 TESTE  usuarioPreenchido_ComTexto_DevolveVerdadeiro
2 inicio
3     conferir_verdadeiro( usuarioPreenchido("joao") )
4 fimteste
```

É o TODO 0, e ele já vem escrito EM C# no seu arquivo — abra e compare com o Portugol aqui de cima, linha por linha. Os outros dezessete têm exatamente esta forma; o que muda em cada um é só o miolo: o valor esperado, e a chamada do método.

```
1 1. leia a REGRA no comentario acima do TODO
2 2. o valor que a regra manda dar -> primeiro parametro
3 3. a chamada do metodo          -> segundo parametro
4 4. devolve SIM/NAO? -> conferir_verdadeiro / conferir_falso
5     devolve numero ou texto? -> conferir_igual
```

Os dezessete, em ordem

TODO	Nome do teste	Confere	Escrito	Barra
1	UsuarioPreenchido_ComTextoVazio_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
2	SenhaPreenchida_ComTextoVazio_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
3	UsuarioDisponivel_ComUsuarioNovo_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
4	UsuarioDisponivel_ComUsuarioJaCadastrado_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
5	SenhasConferem_ComAsDuasIguais_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
6	SenhasConferem_ComSenhasDiferentesDoMesmoTamanho_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
7	SenhaProibida_ComAdmin_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
8	SenhaProibida_ComSenac_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
9	SenhaProibida_ComSenhaComum_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
10	CaracteresIguaisSeguidos_SemNenhumaRepeticao_DevolveUm	igual	☐	☐ ✓ ☐ ✗
11	CaracteresIguaisSeguidos_ComDoisNoComeco_DevolveDois	igual	☐	☐ ✓ ☐ ✗
12	CaracteresIguaisSeguidos_ComTresNoFim_DevolveTres	igual	☐	☐ ✓ ☐ ✗
13	SemRepeticaoDemais_ComSenhaVariada_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
14	SenhaDiferenteDoUsuario_ComSenhaIgualAoUsuario_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
15	PodeCriarConta_ComTudoCerto_DevolveVerdadeiro	verdadeiro	☐	☐ ✓ ☐ ✗
16	PodeCriarConta_ComTudoEmBranco_DevolveFalso	falso	☐	☐ ✓ ☐ ✗
17	MensagemDoErro_ComUsuarioEmUsoESenhasDiferentes_ReclamaDoUsuario	igual	☐	☐ ✓ ☐ ✗

Os quatro que merecem atenção

Não são os mais difíceis de descobrir — são os mais fáceis de ESCREVER errado. Um Assert mal escrito fica vermelho por culpa sua, e você perde tempo procurando defeito onde não tem.

TODO	O cuidado
6 (SenhasConferem)	As duas senhas têm SEIS caracteres cada e são completamente diferentes. É de propósito — não troque os valores
10 a 12 (CaracteresIguaisSeguidos)	Devolve um número, então é a conferência de IGUAL e o esperado vem primeiro. Os três textos são curtos: conte os iguais seguidos na mão antes de escrever
12 (CaracteresIguaisSeguidos)	Repare que os três "1" ficam no FIM de "ab111". Também é de propósito
17 (MensagemDoErro)	Use a CONSTANTE do topo do arquivo, nunca o texto à mão. E repare nos tamanhos: "abc12" tem cinco, "xyz789" tem seis



Vermelho no 17 pode ser você

Antes de anotar o método como defeituoso, releia a sua própria conferência. Metade dos vermelhos de uma primeira noite de teste é o teste, não o programa.

Antes de fechar o Visual Studio

☐	Confira
☐	Os 18 aparecem na lista do Test Explorer — se algum sumiu, faltou a marca de teste em cima
☐	Nenhum está CINZA. Cinza é "não rodou", e quase sempre é a solução que não compila
☐	O Assert de texto usa a CONSTANTE, e não a frase digitada à mão
☐	Nenhum teste tem dois Assert
☐	A folha de palpites tem a terceira coluna preenchida
☐	Você anotou o Expected e o Actual de cada vermelho

Todo vermelho vai para a folha de achados da Parte 9 ANTES de qualquer conserto. Registrar primeiro, consertar depois — a Parte 9.4 explica por quê.

PARTE

9

Achou o vermelho, e agora

- Descrever o achado: regra × código × diferença
- A folha de achados
- Palpite × prova — a conta do fim da noite
- Por que hoje ninguém conserta nada

Como se descreve um achado

"A garantia está errada" não serve para ninguém. Quem vai consertar precisa saber o QUÊ, com QUAL VALOR, e COMPARADO A QUÊ. Use sempre este formato:

```
A REGRA diz: a garantia vale ate o 90o dia, inclusive
O CODIGO diz: if (dias < 90) return true;
A DIFERENCA: NaGarantia(90) da falso, esperado verdadeiro
```

Repare no que NÃO tem aí: nenhum julgamento sobre quem escreveu, e nenhum palpite sobre a correção. Só o que a regra manda, o que o código faz, e o valor exato que mostra a diferença. Isso é um relatório de defeito em miniatura — o modelo completo vem na Aula 3.



O valor exato é o que vale mais

Sem o "NaGarantia(90)" ninguém consegue reproduzir. Um achado que não pode ser reproduzido é uma reclamação, não um defeito.

Folha de achados

#	Teste vermelho	Expected	Actual	A regra dizia
1				
2				
3				
4				
5				
6				
7				
8				

De novo: oito linhas é o tamanho da folha, não a resposta.

Palpite × prova

Agora ponha a folha de palpites (Parte 7) do lado da folha de achados (Parte 9) e preencha:

Acertei

Suspeitei lendo, e a barra confirmou.

Passou batido

A barra achou, e eu não tinha visto.

Acusei sem razão

Suspeitei, e o teste passou.

A coluna do meio é a mais importante da noite, e quase todo mundo tem alguma coisa nela. Ela mostra, com número, por que ler o código não substitui rodar o teste — e é o argumento inteiro da UC11 numa folha de papel.



A terceira coluna também ensina

Acusar um método que estava certo é o que acontece quando a gente confia no olho. Num time, esse palpite viraria um chamado aberto à toa — e alguém perderia meio dia.

Como se conserta, e a ordem que não se inverte

No fim da noite o professor conserta dois ou três no projetor, e você acompanha. Consertar é fácil — o difícil é consertar na ORDEM certa. Estes quatro passos são o trabalho, e pular qualquer um deles custa caro:

#	O passo	O que acontece se você pular
1	REGISTRAR o achado antes de mexer: regra, código, diferença	O defeito some sem nunca ter existido no papel. Ninguém aprende com o que não foi escrito
2	CORRIGIR uma coisa só, e a menor possível	Consertando duas de uma vez, você não sabe qual das duas resolveu
3	RODAR DE NOVO aquele teste, e vê-lo ficar verde	Você acha que consertou. Achar não é saber
4	RODAR A SUÍTE INTEIRA, e conferir que nada mais mudou de cor	Você não sabe se a sua correção quebrou outro método. Isso tem nome: regressão



O passo 4 é o que quase todo mundo pula

E é justamente o assunto da quinta-feira, quando você vai ver uma correção que conserta dois casos e quebra três. Hoje o placar só sobe: 12 verdes, 13, 14. Guarde essa sensação — quinta ela vai ser desmentida.

PARTE

10

Página de socorro

- CS0246: o nome Cadastro não existe
- O teste não aparece no Test Explorer
- Vermelho que não é defeito

Os três problemas de toda primeira noite

O que aparece	O que é	Como resolver
CS0246 — o compilador diz que não conhece o nome Cadastro	O projeto de teste não enxerga a tela	Falta, no topo, a linha que liga ao projeto testado — OU falta a referência de projeto (Parte 6.3)
Escrevi o teste e ele não aparece na lista	O Test Explorer só lista o que compilou	Ctrl+Shift+B. Se continuar sumido, falta a marca de teste em cima do método
O teste aparece CINZA	Ele nem chegou a rodar	A solução não compila. Veja a Lista de Erros — o problema é em outro arquivo
Expected:<Este usuario ja esta em uso.> Actual:<...>	Você digitou a frase à mão e ela saiu diferente	Use a constante USUARIO_EM_USO do topo do arquivo — um ponto final a menos já reprova
Erro de pacote NuGet não encontrado	A máquina não baixou os pacotes do MSTest	Chame o professor. Precisa de internet uma vez só, e não é culpa sua
A conferência de igualdade falhou num método que eu li e está certo	Provavelmente é a sua conferência	Confira a ORDEM: esperado primeiro, obtido depois. E confira o valor esperado contra a Parte 7.1

O que levar desta noite

Um programa que compila e roda pode estar errado.

Defeito e a linha no código. Falha e o que o usuário vê.

O esperado sai da REGRA, nunca do programa.

Verde só prova o que você escreveu.

O programa pode recusar CERTO e explicar ERRADO.

Você acabou de virar testador

Um mês inteiro você escreveu programa. Hoje você recebeu um pronto, que compila, que roda, e que mente em alguns lugares — e a sua ferramenta para provar isso não foi ler com atenção. Foi um programa que confere outro programa.

A parte que fica não é o MSTest: ferramenta se troca. O que fica é a pergunta que você fez dezoito vezes esta noite — "o que este método TEM de fazer, e como eu provo que ele faz?". Ela é a mesma em qualquer linguagem e em qualquer ferramenta.

E fica também o número da coluna do meio: quantos defeitos passaram pelos seus olhos sem que você visse. Não é vergonha nenhuma — é o motivo de esta Unidade Curricular existir.